Accès distant sécurisé avec SSH

Mohamed KHALED

Objectifs

01 Expliquer le rôle de SSH

02 Installer et démarrer le service SSH

03 Renforcer la sécurité du serveur SSH

04 Activer et configurer un pare-feu

Piloter une machine qu'on ne voit pas

Le SSH (pour Secure Shell)

- C’ est un protocole de communication informatique qui permet de se connecter à une machine distante de manière sécurisée.
- En termes simples, c'est un outil qui permet de prendre le contrôle d'un ordinateur (souvent un serveur) situé n'importe où dans le monde, comme si vous étiez assis devant lui, tout en garantissant que personne ne peut intercepter vos échanges.

Pourquoi est-ce "sécurisé" ?

Avant l'arrivée du SSH, on utilisait des protocoles comme **Telnet**. Le problème était que tout ce qui transitait (y compris vos mots de passe) circulait "en clair". N'importe qui sur le réseau pouvait lire vos données.

Le SSH résout ce problème avec trois piliers :

- Chiffrement : Toutes les données échangées entre votre ordinateur et le serveur sont chiffrées. Si quelqu'un les intercepte, il ne verra qu'une suite de caractères illisibles.
- Authentification : Le SSH s'assure que vous êtes bien la personne autorisée à accéder au serveur (via un mot de passe ou, plus couramment, une clé SSH).
- Intégrité : Le protocole vérifie que les données n'ont pas été modifiées ou corrompues pendant leur transfert.

Rappel : client, serveur et service réseau

Terme	Définition simple	Exemple concret
Serveur	Une machine (ou un programme) qui rend un service et attend les demandes.	Le serveur SSH attend qu'on se connecte à lui.
Client	La machine (ou le programme) qui se connecte au serveur pour demander le service.	Votre PC, depuis lequel vous lancez la commande ssh.
Service / daemon	Un programme qui tourne en permanence en arrière-plan et écoute les demandes.	Le service sshd (le « d » signifie daemon).
Port	Un numéro qui identifie un service sur une machine, comme un numéro de porte.	SSH utilise par défaut le port 22.
Adresse IP	L'adresse unique d'une machine sur le réseau, comme une adresse postale.	192.168.1.50

Le protocole SSH : ce qui se passe vraiment

Quand vous lancez une connexion SSH, voici la chaîne d'événements, simplifiée :

1. Le client (votre PC) contacte le serveur sur son adresse IP, sur le port 22.
2. Le serveur et le client négocient un chiffrement : ils se mettent d'accord sur un « langage secret » que personne d'autre ne comprend.
3. Le serveur vérifie qui vous êtes (authentification) : soit par mot de passe, soit par clé.
4. Une fois authentifié, vous obtenez un terminal (shell) sur le serveur. Tout ce que vous tapez part chiffré, tout ce qui revient est chiffré.

Les deux façons de prouver son identité

Par mot de passe

La plus simple à comprendre : vous tapez le mot de passe du compte sur le serveur.

Inconvénient : un mot de passe peut être deviné, volé, ou attaqué par essais répétés (force brute). Sur un serveur exposé à Internet, c'est risqué.

Par clé SSH (la méthode professionnelle)

On génère une paire de clés : une clé privée (qui reste secrète sur votre PC, comme la clé de votre maison) et une clé publique (qu'on dépose sur le serveur, comme la serrure).

Le serveur pose une « énigme » que seule votre clé privée peut résoudre. Si la réponse est bonne, vous entrez sans jamais transmettre de mot de passe sur le réseau.

Installer et gérer le service SSH

```
# Mettre à jour la liste des paquets disponibles
sudo apt update

# Installer le serveur OpenSSH (le service sshd)
sudo apt install openssh-server -y

# Vérifier que le service tourne
sudo systemctl status ssh

# Démarrer / arrêter / redémarrer le service
sudo systemctl start ssh
sudo systemctl restart ssh

# Activer le démarrage automatique au boot
sudo systemctl enable ssh
```

systemctl est l'outil qui pilote les services (les daemons) sous Linux moderne. status affiche l'état (actif/inactif), restart relit la configuration, enable garantit que le service redémarrera tout seul si la machine reboote — indispensable sur un serveur.

Connaître l'adresse de la machine

```
ip a          # affiche toutes les interfaces et adresses IP
hostname -I   # affiche juste la (les) adresse(s) IP
whoami        # affiche le nom de l'utilisateur courant
```

Se connecter à un serveur distant

```
# Syntaxe générale : ssh utilisateur@adresse_ip
ssh mohamed@192.168.1.50

# Se connecter sur un port différent (ex. 2222) avec l'option -p
ssh -p 2222 mohamed@192.168.1.50

# Quitter la session distante
exit
```

La structure utilisateur@adresse se lit « tel utilisateur SUR telle machine ». L'option -p (p minuscule) précise le port quand il n'est pas le 22 par défaut. Pour fermer la connexion, on tape exit, ce qui vous ramène sur votre propre machine.

Les types de clés SSH:

Il existe plusieurs types de clés SSH, qui diffèrent principalement par l'**algorithme mathématique** utilisé pour les générer. Choisir le bon algorithme est important pour garantir un bon équilibre entre la sécurité et la performance.

1. Ed25519 (Le choix recommandé)

C'est le standard moderne et le plus conseillé aujourd'hui.

- **Pourquoi ?** Il est extrêmement sécurisé, très rapide à générer et à utiliser, et les clés sont plus courtes, ce qui les rend plus faciles à manipuler.
- **Compatibilité :** Très largement supporté par les serveurs et clients modernes (OpenSSH 6.5+).
- **Usage :** C'est le choix par défaut pour toute nouvelle configuration.

2. RSA (Le classique)

C'est l'algorithme historique le plus utilisé.

- **Pourquoi ?** Il est compatible avec pratiquement n'importe quel système, même très vieux.
- **Attention :** Pour être considéré comme sûr aujourd'hui, une clé RSA doit avoir une **taille minimale de 3072 ou 4096 bits**. Les clés RSA de 1024 ou 2048 bits sont désormais jugées trop faibles face à la puissance de calcul actuelle.
- **Inconvénient :** La génération et l'utilisation sont plus lentes que Ed25519, et les clés sont beaucoup plus longues.

Les types de clés SSH:

3. ECDSA (Elliptic Curve Digital Signature Algorithm)

Basé sur la cryptographie sur les courbes elliptiques.

- **Pourquoi ?** Il offre une très bonne sécurité avec des clés beaucoup plus courtes que RSA.
- **Inconvénient :** Il a été au cœur de débats sur la confiance (implémentation par la NSA), ce qui pousse beaucoup d'experts à lui préférer **Ed25519** quand c'est possible. Il nécessite également de choisir une courbe spécifique (souvent `nistp256`).

Générer et déployer une paire de clés

```
# Sur le CLIENT : générer une paire de clés (type ed25519, moderne et sûr)
ssh-keygen -t ed25519 -C "srv-debian"

# Copier la clé PUBLIQUE vers le serveur (méthode simple)
ssh-copy-id mohamed@192.168.1.50

# Se reconnecter : plus besoin de mot de passe
ssh mohamed@192.168.1.50
```

ssh-keygen crée deux fichiers dans le dossier ~/.ssh/ : id_ed25519 (la clé privée, à NE JAMAIS partager) et id_ed25519.pub (la clé publique).

L'option -t choisit le type de clé, -C ajoute un commentaire pour s'y retrouver. ssh-copy-id automatise le dépôt de la clé publique dans le fichier ~/.ssh/authorized_keys du serveur.

Les permissions exigées par SSH

```
chmod 700 ~/.ssh                # le dossier
chmod 600 ~/.ssh/id_ed25519     # la clé privée
chmod 644 ~/.ssh/id_ed25519.pub  # la clé publique
chmod 600 ~/.ssh/authorized_keys  # côté serveur
```

SSH refuse d'utiliser une clé privée lisible par d'autres utilisateurs (erreur « UNPROTECTED PRIVATE KEY FILE »). Des droits trop ouverts sur ~/.ssh ou authorized_keys sont une cause très fréquente de « Permission denied (publickey) ».

Le fichier de configuration du serveur SSH

Tout le comportement du serveur se règle dans un seul fichier : /etc/ssh/sshd_config. On l'édite avec sudo (car il appartient à root).

sudo nano /etc/ssh/sshd_config

Les directives (réglages) les plus importantes pour la sécurité :

Directive	Rôle	Valeur recommandée
Port 22	Le port d'écoute du service.	Le changer (ex. 2222) limite les attaques automatiques.
PermitRootLogin	Autorise ou non la connexion directe en root.	no (on se connecte en utilisateur normal puis sudo).
PasswordAuthentication	Autorise la connexion par mot de passe.	no une fois les clés en place.
PubkeyAuthentication	Autorise la connexion par clé.	yes.

Après chaque modification du fichier, il faut redémarrer le service pour qu'il relise sa configuration :

sudo systemctl restart ssh

Le fichier ~/.ssh/config : se simplifier la vie

```
# Sur le CLIENT : créer ou éditer le fichier de configuration
nano ~/.ssh/config

# Contenu : un bloc par serveur
Host srv-debian
    HostName 192.168.1.50
    User mohamed
    Port 2222
    IdentityFile ~/.ssh/id_ed25519

# Désormais, la connexion se résume à :
ssh srv-debian
```

Chaque bloc Host crée un raccourci : plus besoin de retenir l'IP, l'utilisateur, le port ni la clé. scp et sftp profitent aussi de ces alias (scp fichier srv-debian:~). C'est la configuration du CLIENT (~/.ssh/config), à ne pas confondre avec celle du SERVEUR (/etc/ssh/sshd_config).

Exécuter une commande sans ouvrir de session

```
ssh mohamed@192.168.1.50 "df -h"
ssh srv-debian "sudo systemctl restart nginx"
```

Boîte à outils de diagnostic

```
# Côté CLIENT : comprendre pourquoi la connexion échoue
ssh -v mohamed@192.168.1.50    # mode verbeux (-vv, -vvv = encore plus)
ping 192.168.1.50              # la machine répond-elle sur le réseau ?
```

Côté serveur : observer le service

```
sudo systemctl status ssh      # le service tourne-t-il ?
sudo ss -tlnp | grep ssh       # sur quel port écoute-t-il ?
sudo journalctl -u ssh -f      # suivre les logs en direct
sudo sshd -t                   # tester la syntaxe de sshd_config
who                            # qui est connecté en ce moment ?
last                           # historique des connexions
```

Réflexe pro : ssh -v côté client et journalctl -u ssh côté serveur montrent les deux faces de la même connexion. sshd -t est indispensable AVANT de redémarrer le service après une modification de sshd_config : une erreur de syntaxe pourrait vous enfermer dehors.

Activer et configurer le pare-feu UFW

1. Installation

UFW n'est pas installé par défaut sur toutes les versions de Debian. Installez-le avec :

sudo apt update && sudo apt install ufw -y

2. Configuration de base (Politiques par défaut)

Avant d'ouvrir des ports, définissez la politique de sécurité par défaut : tout bloquer en entrée et tout autoriser en sortie.

sudo ufw default deny incoming && sudo ufw default allow outgoing

3. Gestion des règles de ports

Vous pouvez autoriser des services par leur nom ou par leur numéro de port.

AVANT d'activer : autoriser SSH (sinon on se coupe l'accès !)

sudo ufw allow ssh # ou sudo ufw allow 22/tcp

sudo ufw allow 2222/tcp # si le port a été changé

4. Supprimer une règle : Il est souvent plus facile de supprimer par numéro de ligne. Affichez les règles avec les numéros, puis supprimez :

sudo ufw status numbered

sudo ufw delete [numéro]

5.Activer le pare-feu et vérifier

sudo ufw enable

sudo ufw status verbose

Activer et configurer le pare-feu UFW

6.Affiner les règles:

-Autoriser une IP spécifique

Si vous voulez autoriser l'accès à votre machine uniquement depuis une adresse IP de confiance :

sudo ufw allow from 192.168.1.50

-N'autoriser SSH que depuis le réseau local

sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp

sudo ufw delete allow 22/tcp # supprimer une règle

Règle d'or : on autorise TOUJOURS le port SSH avant ufw enable, sous peine de perdre la main sur un serveur distant.

ufw limit ralentit les attaques par force brute en bloquant temporairement les IP qui multiplient les tentatives.

sudo ufw limit 22/tcp

Comparaison avec le monde Windows

Besoin	Sous Windows	Sous Linux
Prendre la main à distance	Bureau à distance (RDP), interface graphique.	SSH, en ligne de commande.
Protocole / port	RDP, port 3389.	SSH, port 22.
Authentification renforcée	Compte AD, MFA.	Clés SSH, désactivation du mot de passe.
Pare-feu intégré	Pare-feu Windows Defender.	UFW (surcouche de iptables).
Expérience	On voit le bureau, la souris.	On voit un terminal texte, plus léger et plus rapide.

À retenir : RDP montre un bureau graphique, SSH donne un terminal texte. SSH consomme beaucoup moins de ressources réseau et c'est le standard absolu pour administrer des serveurs Linux.

Pannes fréquentes sur le terrain

Symptôme à l'écran	Cause la plus probable	Piste de résolution
Connection refused	Le service SSH n'est pas démarré, ou le port est faux.	Vérifier systemctl status ssh et le port utilisé.
Connection timed out	Mauvaise IP, machine éteinte, ou pare-feu qui bloque.	Vérifier l'IP (ip a), le réseau, et ufw status.
Permission denied (publickey)	La clé n'est pas acceptée et le mot de passe est désactivé.	Vérifier le dépôt de la clé publique et les droits du fichier.
Permission denied, please try again	Mauvais mot de passe ou mauvais utilisateur.	Vérifier le nom d'utilisateur et le mot de passe.
Host key verification failed	L'identité du serveur a changé (réinstallation).	Nettoyer la ligne concernée dans ~/.ssh/known_hosts.

Vrai ou Faux ?

1. La clé privée doit être copiée sur tous les serveurs auxquels je veux me connecter.
2. SSH chiffre les données qui circulent entre le client et le serveur.
3. Telnet est plus sûr que SSH.
4. Le port par défaut de SSH est le 3389

Lecture de commande

Que fait la commande suivante ? Décomposez chaque partie :

ssh -p 2222 admin@10.0.0.12

Le diagnostic minute

Un collègue lance une connexion et obtient le message « Connection timed out ». Citez deux causes possibles et la première commande que vous taperiez pour enquêter.

Réflexion sécurité

Pourquoi désactive-t-on souvent la connexion directe en tant que root via SSH ? Que fait-on à la place pour exécuter une commande administrateur ?